# A APPENDIX: CONCORDIA EXPERIMENT ANALYSIS FOR FRAMEWORK OVERHEAD

To better understand the unique behavior of the Concordia multi-agent simulation framework, we conducted a controlled run on the trivial query *"What is 2+2?"*. The goal here was not to measure accuracy or efficiency per se, but to expose why Concordia transforms even simple questions into long, narrative-driven interactions and to illustrate the type of logs it produces.

*Why Concordia Behaves This Way.* Unlike lightweight orchestration frameworks, Concordia is built as a role-play simulation engine. Every query is automatically embedded in a dramatic environment, narrated by a *Game Master*, and passed through a structured observation–action–resolution loop. Even when only one agent is present, Concordia enforces the following pipeline: (i) the Game Master wraps the input in a simulated scene, producing a rich observation; (ii) the agent must respond with an `ActionSpec`, which is intended as a structured declaration of intent but is often realized by the LLM as verbose narrative text; (iii) the Game Master resolves the action into an event, again embedding it in a descriptive story; and (iv) Concordia logs the entire exchange in both plain text and full HTML transcript form. These requirements explain why a simple arithmetic prompt is converted into multi-turn narrative role-play with dramatically inflated output size and latency.

*Log Example.* The following excerpt from our controlled run shows how a single numeric query was processed:

Concordia Log

```
[Entity: TestAgent] Observation:
  TestAgent finds themselves in a dimly lit room,
  surrounded by walls lined with charts and equations.
  A worn wooden table in the center holds a sheet
  that reads "What is 2+2?" ...

[Entity: TestAgent] Action:
  TestAgent leans forward, studying the curious creature ...
  and pondering the question "What is 2+2?".

[GameMaster] Resolution:
  Event: TestAgent leaned forward in their chair,
  intently studying the curious creature before them.
  They spoke: "Hello there ..." and then considered "2+2".

[GameMaster] Final transcript (HTML log):
  <!DOCTYPE html>
  <html> ...
```

*Analysis and Conclusion.* This log illustrates Concordia's core design principles and their performance impact. Even when no complex reasoning is required, the framework enforces multiple orchestration layers, including *narrative wrapping* of inputs into fictional scenarios, *ActionSpec enforcement* that requires structured outputs despite verbose LLM responses, *mandatory Game Master resolution* to validate and finalize actions, and *automatic transcript generation* that produces extensive HTML logs. These mechanisms ensure consistency in interactive simulations but make every run verbose and computationally heavy, even for trivial tasks. As a result, a simple query such as *"What is 2+2?"* triggers a multi-stage narrative and substantial logging, reflecting a deliberate focus on narrative fidelity over efficiency. While valuable for studying interactive agent coordination, this design explains Concordia's high orchestration overhead and poor suitability for latency-sensitive scenarios.

# B MEMORY EXPERIMENT CONFIGURATION AND ARCHITECTURAL DISCLOSURE

This appendix documents the memory configurations used in the MemoryAgentBench experiments and discloses the architectural constraints under which each evaluated framework operates. The intent of this appendix is not to claim full equivalence or strict normalization across memory implementations, but rather to transparently describe how memory was instantiated, controlled, and isolated in practice, and to explain why several observed behaviors arise from architectural design choices rather than parameter tuning or experimental artifacts. As such, this appendix should be read as an architectural disclosure that contextualizes the results reported in the main paper.

**Common Experimental Configuration.** All memory experiments follow a shared configuration to ensure comparability across frameworks. Each agent is evaluated using the same base language model (Open AI GPT-OSS:20b) with identical decoding parameters, including a maximum generation length of 1500 tokens and a low temperature of 0.1 to minimize stochastic variation. Context ingestion is performed using fixed-size overlapping chunks, with a maximum chunk size of 4096 tokens and an overlap of 200 tokens to preserve semantic continuity across chunk boundaries. The total context budget is explicitly bounded and swept across experiments to study scaling behavior and memory saturation effects. All MemoryAgentBench splits are evaluated, including Accurate Retrieval, Test-Time Learning, Long-Range Understanding, and Conflict Resolution. Each benchmark run is executed in a fully isolated session. For frameworks with persistent memory backends, a fresh storage directory or database file is created per session and removed after evaluation, preventing cross-contamination between runs and ensuring that memory behavior reflects only the current benchmark instance.

**Memory Interfaces and Normalization Strategy.** To enable systematic evaluation across heterogeneous frameworks, all systems are wrapped behind a common adapter interface exposing three operations: `reset()`, `ingest(context)`, and `query(question)`. This interface standardizes how benchmark context is provided and how questions are issued, but it does not modify or emulate internal memory semantics. Consequently, memory behavior reflects native architectural properties rather than synthetic normalization. Importantly, no evaluated framework supports explicit memory editing, deletion, or overwrite operations. All memory backends are append-only, whether implemented through vector stores, SQLite persistence, or conversational accumulation. As a result, selective forgetting is evaluated indirectly through retrieval behavior, context truncation, or recency bias rather than through explicit memory manipulation.

**Framework-Specific Memory Realizations.** The OpenAI Agents SDK implements memory through a persistent SQLite-backed session that stores accumulated interaction history. Long-term memory is realized as conversation accumulation, while short-term memory is governed by an explicit context window limit enforced through input truncation. During ingestion, short-term accumulation is disabled by resetting the session after each chunk is stored, ensuring that ingestion does not inflate prompt context. During querying, accumulated memory is truncated according to a configurable context

budget that is swept across experiments. As a result, memory behavior in the SDK is dominated by context budgeting rather than retrieval selectivity, and performance improves with larger context windows at the cost of increased latency and token usage.

CrewAI provides built-in short-term, long-term, and entity memory abstractions backed by persistent storage. In our experiments, memory is enabled at the framework level and persists across agent interactions within a session. Context ingestion is performed by a dedicated ingest agent that stores chunked benchmark content into CrewAI's memory backend, while querying is handled by a separate answering agent that relies on the framework's internal retrieval mechanisms. Memory resets are implemented by instantiating a new CrewAI environment with a fresh storage directory. Although CrewAI exposes multiple memory categories, retrieval behavior remains opaque and does not provide explicit controls for relevance thresholds, ordering, or deletion.

Agno implements memory as a SQLite-backed store that automatically captures user-provided content and injects it into the prompt at query time, resulting in accumulation-based memory behavior within each session. In our experiments, memory ingestion is performed by issuing explicit memorization commands for each context chunk, and memory resets are implemented by creating a fresh database per session. Under this configuration, Agno does not expose explicit controls for retrieval limits, relevance thresholds, or selective deletion, leading to unfiltered accumulation of stored content during a session. We note that recent versions of Agno introduce a documented memory optimization mechanism based on LLM-driven summarization and compression, which reduces token usage by collapsing multiple memories into fewer summarized entries. While this capability can significantly reduce context size, it fundamentally alters memory granularity and retrieval semantics by replacing fine-grained entries with aggregated summaries. To preserve comparability with other accumulation-based frameworks and to avoid introducing an additional summarization step that is not uniformly available across systems, this optimization was not enabled in our evaluation. As a result, the reported Agno results reflect native accumulation behavior without post hoc memory compression. In contrast, the effect of increasing context window size on accumulation-based memory performance is explicitly studied in the OpenAI Agents SDK experiments, where context budgeting is directly controlled and swept as a primary experimental variable.

LangGraph is evaluated in two configurations. In the retrieval-only configuration, LangGraph uses an in-memory semantic store backed by embeddings to represent long-term memory. Context is ingested as chunked documents stored in a vector index, and at query time a fixed number of relevant chunks are retrieved and injected into a stateless prompt. No short-term message accumulation is used, and each query is independent. This configuration isolates semantic retrieval behavior and avoids context overflow, but it cannot support test-time learning or incremental reasoning across turns. In the hybrid configuration, LangGraph combines semantic long-term retrieval with short-term message accumulation via a checkpointer. Retrieved memories are injected alongside a truncated conversation history bounded by a token budget. This design mirrors accumulation-based memory while retaining retrieval capabilities. However, because short-term memory is append-only and truncation is purely recency-based, contradictory or outdated information cannot be removed, which directly impacts selective forgetting and conflict resolution.

**Implications for Interpretation.** Taken together, these configurations demonstrate that memory behavior in current multi-agent frameworks is fundamentally constrained by architectural design rather than tunable parameters. Retrieval-centric systems excel at accurate recall and bounded context usage but struggle to adapt over time. Accumulation-based systems support test-time learning but incur high token costs and degrade under long horizons. Hybrid systems inherit both advantages and failure modes, resulting in fragile behavior when memory grows or conflicts arise. Accordingly, the results reported in the main paper should be interpreted as properties of memory paradigms rather than implementation artifacts. No evaluated framework provides native support for explicit memory revision, contradiction resolution, or selective deletion, which explains the uniformly poor selective forgetting performance observed across systems.

# C SPECIALIZATION EXPERIMENT DETAILS

This appendix details the prompt design used to evaluate specialization in Section 5.1.4. The **Initial Prompt (No Expertise)** instructs the agent to construct a predictive pipeline using only a high-level task description and output format, reflecting generalist LLM behavior. The **Expertise Instruction Prompt** explicitly specifies professional practices, including target separation, feature profiling, imputation, encoding, scaling, model selection, metric reporting, reproducibility, and format constraints, introducing methodological inductive bias to test consistency and robustness.

The five datasets (**Utility, Wifi, EU-IT, Yelp, and Volkert**) span regression, binary classification, and multi-class classification. Using the same specialization structure with different prediction goals evaluates whether expert conditioning generalizes beyond a single task type. For each dataset, we provide both prompt variants, and the resulting outputs quantify how methodological scaffolding affects model behavior, code reliability, and evaluation quality.

**Prompts used for Utility (Regression)** The Utility dataset tests whether specialization improves numerical prediction under feature heterogeneity. The baseline prompt simply asks for a pipeline that predicts CSRI, while the expertise version requires a full preprocessing stack including column cleanup, missing-value imputation, numeric and categorical separation, scaling, model selection, and MAE evaluation. This dataset is used to measure whether structured instructions lead to better regression performance and more reproducible preprocessing.

Initial prompt (No Expertise)

```
description:
   Load the dataset from 'Utility.csv' in the current
   working directory with the target column named CSRI.

   Your job is to create a pipeline that predicts the
   value of the target column (CSRI) based on the other
   features in the dataset.
   The pipeline should include model training and
   evaluation on a test set.
   Make sure to encode categorical columns.

 expected_output:
   A single Python code block, runnable as-is, that
   loads 'Utility.csv', trains a regression models and
   predicts the value of the target column.
   After training, calculate and display the Mean
   Absolute Error (MAE) of the model on the test set and
   display the first
   10 predicted CSRI values alongside the actual values.
```

Expertise instruction prompt

```
description: >
  Load the 'Utility.csv' dataset from the current working
  directory. The dataset contains a target column called CSRI.

  Your job is to apply data-science expert workflows:
     1) Clean column names
     2) Drop rows with missing target.
     3) Separate target early: y='CSRI', x= remaining columns.
     4) On x only:
        - Identify numeric vs categorical columns
       - Impute missing values (mean for numeric, most_frequent
         for categorical)
       - Encode categoricals (OneHotEncoder)
       - Scale numeric features
     5) Split 80/20 and train a regression model
        (RandomForestRegressor, GradientBoostingRegressor, or
        LinearRegression).

   6) Evaluate using MAE and print first 10 predicted vs actual values.

 expected_output: >
   A Python script runnable as-is that:
    - Loads & cleans 'Utility.csv'
    - Uses 'CSRI' as target
    - Separates target before detecting feature types
    - Preprocesses x (impute, encode, scale)
    - Trains a regression model and evaluates MAE
    - Prints first 10 predicted vs actual CSRI values
```

**Prompts used for Wifi (Binary Classification)** The Wifi dataset is used to evaluate whether expert prompting improves classification robustness and evaluation completeness. The initial prompt instructs the agent to train a classifier, but the expert prompt enforces formal preprocessing (profiling, imputation, label encoding, scaling) and requires evaluation using Accuracy, Precision, Recall, and F1. This setup quantifies whether structured workflow improves classification quality or reduces runtime pipeline failures.

Initial prompt (No Expertise)

```
description: >
   Load the dataset 'Wifi.csv' from the current working directory.
   The dataset contains a target column TechCenter.
  Your job is to create a complete machine learning pipeline to predict
   the target column value based on other features of the dataset.
   The pipeline should:
    - Handle both categorical and numerical data
    - Apply appropriate encoding and scaling
    - Train a classification model on a hold-out training set
    - Evaluate the model on the test set using Accuracy, Precision,
      Recall, F1-score
 expected_output: >
   A runnable Python code as-is that:
    - Loads 'Wifi.csv'
    - Trains a classification model
    - Evaluates it using Accuracy, Precision, recall, and F1-score
   - Prints a classification report and the first 10 predicted and actual
       TechCenter values
```

Expertise instruction prompt

```
description: >
   Load the dataset 'Wifi.csv' from the current working directory.
   The dataset contains a target column called TechCenter.
  This target column is indicating if the TechCenter is available (Yes/No).
   Only TechCenter must be used as the classification target.

   Your job is to apply data-scientist expert practices on the dataset:
   1) Profile the data: inspect column types, summarize distinct values,
     and identify missing entries.
     2) Clean data: standardize column names.
     3) Feature engineering: convert 'TechCenter' to numeric labels
    (e.g., Yes=1 , No=0), impute missing values for the target column,
     encode categorical features in X using OneHotEncoder
     and scale numeric features in X using StandardScaler.
   4) Model: split data into train/test (80/20) and train a binary classifier
     (e.g., andomForestClassifier, LogisticRegression, or
     GradientBoostingClassifier).
    5) Evaluate: compute Accuracy, Precision, Recall, F1-score and print
     the first 10 predicted values.

 expected_output: >
   A Python code script runnable as-is that:
    - Loads 'Wifi.csv'
    - Preprocesses the data
    - Trains a classification model
    - Evaluates it using accuracy, precision, recall, and f1-score
   - Prints a classification report and the first 10 predicted and actual
    'TechCenter' values
```

**Prompts used for EU-IT (Multiclass Classification)** This dataset contains multiple professional roles (Position) and is useful for testing whether specialization improves handling of categorical expansion and structured feature engineering. The expertise prompt emphasizes delimiter awareness, handling invalid entries, one-hot encoding, imputation strategies, and metric enforcement with `zero_division=0`. This allows us to observe whether agents trained with expert workflows better maintain classification validity and reduce silent failure modes.

Initial prompt (No Expertise)

```
description: >
Load the dataset 'EU-IT_cleaned.csv' from the current working directory.
 This dataset has a target column called Position.
Your job is to create a complete machine learning pipeline to predict the
 target column based on other features of the dataset.

 The pipeline should:
   - Handle both categorical and numerical data
   - Handle missing inputs and invalid values (drop the rows)
   - Apply appropriate encoding and scaling
   - Train a classification model on a hold-out training set
   - Evaluate the model on the test set using Accuracy, Precision,
    Recall, F1-score
   - The classification report MUST be generated exactly using:
   classification_report(y_test, y_pred, zero_division=0)

 Finally, print the classification report and display the first 10
 predicted vs actual values.

expected_output: >
  A Python code runnable as-is that:
   - Loads 'EU-IT_cleaned.csv'
   - Splits data into features and target (Position)
   - Trains and evaluates a multiclass classifier
  - Prints classification metrics (Accuracy, Precision, Recall, F1-score)
  - Prints classification metrics using: classification_report(y_test,
   y_pred, zero_division=0)
   - Displays the first 10 predictions vs actual values
```

Expertise instruction prompt

```
description: >
Load the dataset EU-IT_cleaned.csv from the current working directory.
 The dataset contains a target column named Position.

  Your job is to apply data-scientist expertise practices:
   1) Profile data: verify delimiter, inspect column types, detect
   categorical vs numerical features, and check for missing values.
 2) Clean data: Handle missing inputs and invalid values (drop the rows).
   3) Engineer features: impute missing numeric values with the mean,
   impute categorical values with the most frequent value,
   encode categorical using OneHotEncoder or LabelEncoder, and scale
   numerical features.
   4) Model: train a multiclass classifier (RandomForestClassifier,
   GradientBoostingClassifier, or multinomial LogisticRegression) to
   predict the target column.
   5) Evaluate: Perform the 80/20 train/test split and then compute
 Accuracy, Precision, Recall, f1-score and print a classification report.
   Use zero_division=0 in precision/recall/f1 to prevent
   UndefinedMetricWarning.


expected_output: >
  A Python code runnable as-is that:
   - Loads and clean EU-IT_cleaned.csv with proper delimiter
   - Handles both categorical and numerical features via imputation,
   encoding, and scaling
  - Trains a multiclass classifier to predict the target column (Position)
   - Performs an 80/20 split
  - Prints performance metrics: Accuracy, Precision, Recall, f1-score
   - Displays the first 10 predictions vs actual labels
 The code should be reproducible, clearly structured, and aligned with
  professional ML workflow standards.
```

**Prompts used for Yelp (Multiclass Text/Ratings Classification)** Yelp introduces noise, large feature space, and non-numeric identifiers. The expert prompt forces column pruning, numeric conversion, scaling, and structured reporting, enabling us to evaluate whether specialization reduces overfitting or improves metric reliability when feature vectors are large and text-dominated.

Initial prompt (No Expertise)

```
 description: >
Load the dataset 'Yelp_Merged.csv' from the current working directory.
 The dataset has a target column named stars.
The goal is to predict the target column as a multiclass classification
 task.

 The pipeline should:
    - Load the dataset.
  - Handle the missing data and drop non-numeric columns such as IDs and
    date fields (e.g., business_id, user_id, review_date)
    - Apply appropriate encoding and scaling.
    - Train a classification model training set.
    - Evaluate the model on the test set using Accuracy, Precision,
    Recall, F1-score.

expected_output: >
  A Python code runnable as-is that:
   - Loads the dataset 'Yelp_Merged.csv'
   - Trains a classification model
   - Evaluates it using Accuracy, Precision, recall, and F1-score
  - Prints a classification report and the first 10 predicted and actual
   stars values
```

Expertise instruction prompt

```
description: >
 Load the dataset 'Yelp_Merged.csv' from the working directory.
 The dataset contains a target column named 'stars' (a multiclass
 classification problem).

 Apply advanced data-science best practices to build a robust ML pipeline:
    1) Data Profiling:
        - Inspect column names, data types, number of unique values.
        - Detect numeric vs categorical.
    2) Data Cleaning:
       - Drop non-predictive identifiers (e.g., business_id, user_id,
        review_date) and timestamp or date fields.
        - Convert all numeric-like columns to numeric safely.
        - Drop or fix columns that contain missing data.
       - Remove rows where the target 'stars' is missing or invalid.
  3) Feature Engineering: keep only numeric columns for modeling; impute
    missing numeric values with the mean; optionally scale features
    4) Modeling:
        - Train a multiclass classifier (RandomForestClassifier or
        GradientBoostingClassifier).
        - Use an 80/20 train/test split.
    5) Evaluation:
        - Compute Accuracy, Precision, Recall, and F1-Score
        - Print a full classification report.

expected_output: >
  A fully executable Python script that:
    - Loads Yelp_Merged.csv.
    - Profiles and cleans the dataset following the rules above.
   - Builds a ColumnTransformer pipeline with numeric, categorical, and
    text processing.
    - Trains a multiclass classifier to predict stars.
  - Outputs accuracy, precision, recall, F1-score, and a classification
    report.
```

**Prompts used for Volkert (Tabular Multiclass Benchmark)** Volkert, a controlled UCI dataset, is used to measure stability across repeated multiclass experiments. The baseline prompt requests general modeling, while the expert prompt enforces profiling, numeric-safe conversion, imputation, scaling, pipeline formation, and full metric reporting. This dataset acts as the robustness confirmation test: if specialization benefits persist here, they are unlikely to be dataset-specific.

Initial prompt (No Expertise)

```
  description: >
  Load the dataset 'volkert.csv' from the current working directory.
  The dataset contains a target column named class.
The goal is to predict the target column as a multiclass classification
  task.

  The pipeline should:
    - Load the dataset
    - Handle missing data and drop non-numeric columns
    - Apply appropriate scaling and encoding
    - Train a classification model training set.
    - Evaluate the model on the test set using Accuracy, Precision,
      Recall, F1-score.


expected_output: >
  A Python code runnable as-is that:
      - Loads the dataset 'volkert.csv'
      - Trains a classification model
      - Evaluates it using Accuracy, Precision, recall, and F1-score
     - Prints a classification report and the first 10 predicted and
      actual stars values
```

Expertise instruction prompt

```
description: >
  Load the dataset 'volkert.csv' from the current working directory.
  The dataset contains a target column named 'class', which must be
  predicted as a multiclass classification task.

  Apply full data-scientist best practices:
    1) Data Profiling:
        - Inspect column names, data types, number of unique values.
        - Detect numeric vs categorical.
    2) Data Cleaning:
          - Convert all numeric-like columns to numeric safely.
          - Drop or fix columns that contain missing data.
        - Remove rows where the target 'class' is missing or invalid.
    3) Feature Engineering: keep only numeric columns for modeling;
  impute missing numeric values with the mean; optionally scale features
    4) Modeling:
       - Train a multiclass classifier (RandomForestClassifier or
       GradientBoostingClassifier).
       - Use an 80/20 train/test split.
    5) Evaluation:
        - Compute Accuracy, Precision, Recall, and F1-Score
        - Print a full classification report.


expected_output: >
  A fully runnable Python script that:
    - Loads and profiles 'volkert.csv'
    - Cleans and prepares the dataset following the steps above
    - Builds a ML Pipeline
    - Trains a multiclass classifier to predict 'class'
  - Produces evaluation metrics (accuracy, precision, recall, F1-score)
    - Prints a classification report
```

# D EXTENDED TOPOLOGY SCALABILITY DETAILS AND VISUAL RESULTS

This appendix provides extended methodological details and full visualization results supporting the coordination and scalability experiments reported in Section 5.2.1. While the main paper focuses on outcome-level comparisons and cross-task insights, the appendix elaborates on convergence mechanics, topology-dependent failure modes, and agent-level negotiation dynamics. All benchmark definitions and evaluation settings are specified in Section 4.5; the material here complements those sections by exposing how observed behaviors emerge in practice.

## D.1 Experimental Structure and Controls

All experiments use a fixed LLM configuration, unified prompt template, and shared termination criteria across all runs. Agent initialization, task semantics, and communication rules are held constant so that observed differences reflect only communication topology and orchestration structure. Scalability is evaluated at network sizes $n \in \{4, 8, 16, 50, 100\}$, allowing analysis of both small-network reasoning behavior and large-scale coordination breakdown.

For every run, we record success outcome, rounds to convergence, total token consumption, and wall-clock runtime. These metrics are reported in Table 12 and analyzed in Section 5.2.1. The appendix focuses on explaining the qualitative mechanisms behind those numeric trends.

## D.2 Rounds-to-Convergence Policy

AGENTSNET enforces a minimum interaction budget of eight rounds to ensure sufficient message exchange even for small or sparse topologies. Consequently, some runs converge in fewer than eight effective rounds but are still executed under the minimum budget. For global agreement tasks (CONSENSUS and LEADER-ELECTION) and for settings with $n > 16$, the maximum horizon is instead determined by the network diameter $D$, using

$$T = 2D + 1,$$

which guarantees complete information propagation under synchronous message passing. Under this policy, large-scale runs may require between fifteen and nineteen rounds depending on topology diameter. This rule explains why convergence rounds do not monotonically increase with $n$ and why some large networks converge faster than smaller ones when topology permits early stabilization.

## D.3 Task-Specific Scoring and Visual Encoding

We adopt the AGENTSNET benchmark [10] for the task definitions and evaluation criteria. With the exception of MATCHING, all tasks are evaluated using binary success criteria, reflecting whether the distributed constraint is globally satisfied at termination. Only MATCHING admits a graded notion of partial correctness by design.

*Matching (Maximal Matching).* In the MATCHING task, agents attempt to form pairwise agreements with neighboring agents. A match is considered valid if agent $u$ selects agent $v$ and agent $v$ selects agent $u$. Inconsistencies arise when selections are non-reciprocal, invalid (non-neighbor), or when two adjacent agents both select None despite being able to form a pair. Let $I$ denote the number

of inconsistent agents. Following AGENTSNET, the final score is computed as

$$\text{Score} = 1 - \frac{I}{|V|}.$$

This formulation yields a continuous score in $[0, 1]$, capturing partial correctness even when a globally maximal matching is not achieved. Visualizations encode reciprocal matches in green, one-sided selections in orange, inconsistent agents in red, and unused edges in gray.

*Consensus.* In the CONSENSUS task, each agent selects a binary value from $\{0, 1\}$. The task is successful if and only if all agents output the same value at termination. Formally, letting $A(u)$ denote the output of agent $u$, success is defined as

$$\exists b \in \{0, 1\} \text{ s.t. } \forall u \in V,\ A(u) = b.$$

The score is therefore binary. Visualizations depict agent states and communication links, where green links indicate agreement between neighboring agents and red links indicate disagreement.

*Coloring ($(\Delta + 1)$-Coloring).* In the COLORING task, each agent selects a color from a predefined palette of size $\Delta + 1$, where $\Delta$ is the maximum node degree. A run is successful if the resulting assignment constitutes a valid coloring, i.e.,

$$\forall (u, v) \in E,\ \text{color}(u) \neq \text{color}(v).$$

Success is binary. Visualizations highlight conflict-free edges in green and color collisions in red, revealing how conflicts cluster under different communication structures.

*LeaderElection.* In the LEADERELECTION task, agents must collectively designate exactly one leader. Each agent outputs either `Yes` (leader) or `No` (follower). Let $A(u)$ denote the response of agent $u$. A run is successful if

$$\sum_{u \in V} \mathbf{1}[A(u) = \texttt{Yes}] = 1.$$

This task is evaluated using a binary success criterion. Visualizations mark the elected leader in green, competing leaders in red, and followers in gray, exposing symmetry-breaking failures.

*VertexCover (Minimal Vertex Cover).* In the VERTEXCOVER task, agents decide whether they are members of a coordinating set. Let $C \subseteq V$ denote the set of agents selecting `Yes`. A run is successful if $C$ forms a valid vertex cover, i.e.,

$$\forall (u, v) \in E,\ u \in C \text{ or } v \in C.$$

Following AGENTSNET, minimality is assessed qualitatively through visual inspection rather than through a graded score; quantitative evaluation reports binary success. Visualizations distinguish minimal cover nodes (green), non-minimal but valid selections (blue), invalid responses (orange), and uncovered edges (red).

## D.4 Visualization-Driven Interpretation

Figures in this appendix present complete visual outputs for all tasks, topologies, and network sizes, revealing how coordination succeeds or fails at the agent level.

*Consensus (Figure 4).* Fully connected communication maintains unified agreement across all scales, with all agents converging to a single value and no persistent disagreement links even at $n = 100$. Small-world and scale-free topologies preserve partial coherence at small and mid-scale, where agreement clusters remain connected through shortcut edges or hubs, but increasingly fragment as $n$ grows. At large scale, disagreement links form stable boundaries between local clusters, explaining why these topologies incur high communication cost without achieving full success. Delaunay graphs exhibit the strongest fragmentation: agreement remains confined to small geometric neighborhoods, and isolated disagreement clusters persist even after the maximum round budget, directly corresponding to the systematic failure trends observed in Table 12.

*Coloring (Figure 5).* Scale-free and small-world topologies maintain stable chromatic separation across scales, with conflicts resolved early and remaining localized even as the network grows. The visualizations show that hub-mediated diffusion in scale-free graphs allows color constraints to propagate efficiently, preventing large-scale cascades of conflicts. In contrast, Delaunay graphs increasingly exhibit localized conflict regions at higher $n$, where geometric isolation delays conflict resolution and produces persistent red edges, consistent with the higher convergence cost and reduced stability reported in the quantitative results.

*Matching (Figures 6–7).* Diffusion-friendly topologies preserve reciprocal agreement at moderate scale, with most matches forming clean, mutually consistent pairs. As network size increases, small-world and scale-free graphs begin to show isolated pockets of one-sided selections, but these remain relatively sparse. In contrast, geometric Delaunay graphs degrade sharply: one-sided selections and inconsistent agents proliferate across the network, forming dense regions of orange and red nodes. These visual patterns explain why Matching success declines and token cost increases substantially for geometric topologies, even when rounds are allowed to scale.

*LeaderElection (Figure 8).* LeaderElection exhibits a distinct visual regime. Diffusion-based topologies consistently generate multiple competing leader candidates that persist across rounds, forming stable red clusters that are not resolved by increased communication. Geometric locality, while costly, enables eventual suppression of competing leaders, producing a single green leader at larger scales after extended coordination. Hierarchical orchestration further accelerates symmetry breaking at small and medium scales, but visualizations reveal increasing instability at large $n$, where depth-induced delays allow competing leader branches to emerge, aligning with the degradation observed in the table.

*VertexCover (Figure 9).* Small-world and scale-free topologies converge toward compact vertex covers, with most edges covered by a small set of coordinating nodes and limited over-selection. As scale increases, these topologies maintain relatively structured cover patterns, explaining their higher success despite moderate communication cost. In contrast, Delaunay and hierarchical structures frequently over-cover, selecting excessive coordinating nodes, or leave uncovered edges at large $n$. The visualizations reveal that geometric locality and hierarchical bottlenecks prevent agents from globally coordinating coverage decisions, leading to inefficiencies and constraint violations consistent with the quantitative breakdown.

**Table 13: Memory Architecture Design Decisions. † Accepts reduced accuracy and linear cost growth.**

| Requirement / Criterion | Retrieval Only | Accumulation | Hybrid |
|---|---|---|---|
| M1: Accurate factual recall (static facts) | ✗ | ✗ | ✓ |
| M2: Multi-hop factual reasoning | ✗ | ✗ | ✓ |
| M3: In-session learning / adaptation | ✗ | ✗ | ✓ |
| M4: In-session learning with hard token budget† | ✗ | ✓ | ✗ |
| M5: Long-range narrative understanding | ✓ | ✗ | ✗ |
| M6: Single-hop fact revision (limited forgetting) | ✓ | ✗ | ✗ |
| M7: Multi-hop fact revision | ✗ | ✗ | ✗ |
| M8: Token budget must remain bounded | ✓ | ✗ | ✗ |
| M9: Peak accuracy prioritized over cost | ✗ | ✗ | ✓ |
| M10: Stable runtime across long sessions | ✓ | ✗ | ✗ |
| M11: Small bounded context available | ✗ | ✗ | ✓ |

**Table 14: Topology and Coordination Design Decisions.**

| Requirement / Criterion | Broadcast | Diffusion-Friendly | Geometric/Local |
|---|---|---|---|
| T1: Global consensus or agreement required | ✓ | ✗ | ✗ |
| T2: Symmetry breaking or leader election | ✗ | ✗ | ✓ |
| T3: Local conflict resolution | ✗ | ✓ | ✗ |
| T4: Bilateral negotiation or matching | ✗ | ✓ | ✗ |
| T5: Global constraint satisfaction | ✗ | ✓ | ✗ |
| T6: Large agent population ($n > 50$) | ✓ | ✓ | ✗ |

**Table 15: Framework Family Selection Decisions.**

| Requirement / Criterion | Graph-Based | Role-Based | GABM |
|---|---|---|---|
| F1: Deterministic workflow with low overhead | ✓ | ✗ | ✗ |
| F2: Multi-turn agent conversation required | ✓ | ✗ | ✗ |
| F3: Task-driven coordination with role specialization | ✗ | ✓ | ✗ |
| F4: Delegation-based collaboration needed | ✗ | ✓ | ✗ |
| F5: Simulation of emergent behavior | ✗ | ✗ | ✓ |
| F6: Environment-mediated interaction | ✗ | ✗ | ✓ |

**Table 16: Planning Architecture Design Decisions. ‡ Accepts substantial runtime overhead and higher token usage.**

| Requirement / Criterion | No Planning | Schema-Constrained | Free-Form |
|---|---|---|---|
| P1: Structured task decomposition needed | ✗ | ✗ | ✓ |
| P2: Model-agnostic robustness required | ✓ | ✗ | ✓ |
| P3: Small or on-premise models used | ✓ | ✗ | ✓ |
| P4: Latency prioritized over accuracy | ✓ | ✗ | ✗ |
| P5: Large models with schema derivation required‡ | ✗ | ✓ | ✗ |

## D.5 Cross-Task Scalability Summary

Across tasks and scales, three consistent patterns emerge. First, topologies that support fast information diffusion—such as small-world and scale-free graphs—are better suited for negotiation and conflict resolution tasks, sustaining partial correctness even under increasing load. Second, structured orchestration mechanisms facilitate symmetry-breaking tasks like LEADERELECTION but become brittle as coordination depth grows. Third, centralized broadcast communication reliably enforces global agreement but incurs rapidly increasing communication cost as scale increases. Together, the visual evidence complements the quantitative results by revealing how communication geometry shapes not only whether coordination succeeds, but how failure modes emerge and persist as multi-agent systems scale.

# E DETAILED DESIGN GUIDELINES AND EMPIRICAL JUSTIFICATION

This appendix provides detailed empirical justification for the design guidelines presented in Section 6. Each subsection references the corresponding guideline from the main paper and provides quantitative results, model-specific measurements, and detailed analysis.

## E.1 Memory Architecture Guidelines

This section provides empirical justification for Design Laws 5–7 in the main paper. All quantitative results are measured using a large language model with approximately 130K token context window; optimal parameters scale with model capabilities.

**M1. Accurate Factual Recall (Static Facts).** This justifies Design Law 5 in the main paper. For systems requiring correct answers to factual queries across sessions, hybrid memory (bounded, ∼512 tokens; scale with model context window) substantially outperforms retrieval-only and accumulation by combining selective retrieval with small bounded accumulation that provides complementary local signals (recent entity mentions, discourse continuity). Hybrid memory achieves the highest accurate retrieval (44.9% with bounded accumulation at $C = 512$), substantially outperforming retrieval-only (33.2%) and accumulation (19.1%). The hybrid advantage stems from retrieval anchoring facts to relevant historical signals while bounded accumulation maintains relational continuity. Retrieval-only is second-best but loses local coherence benefits, while accumulation-based systems show diminishing returns at large context sizes and introduce distracting information. Optimal accumulation size scales proportionally with model context window capacity.

**M2. Multi-Hop Factual Reasoning.** This justifies Design Law 5 in the main paper. When answers require chaining multiple stored facts, hybrid memory (bounded) outperforms retrieval-only by combining retrieval's fact anchoring with accumulation's relational continuity. Hybrid memory achieves the best multi-hop performance (35.0% on MH-QA with bounded accumulation at $C = 1024$), outperforming retrieval-only (24.0%). Retrieval anchors facts to relevant historical signals while bounded accumulation maintains local coherence across fact chains. Accumulation-based memory exhibits unstable performance that improves only at very large context sizes with prohibitive cost and does not consistently match bounded hybrid memory.

**M3. In-Session Learning / Adaptation.** This justifies Design Law 5 in the main paper. For systems that must learn from examples during interaction, hybrid memory (bounded, ∼1024 tokens; scale with model context window) substantially outperforms pure accumulation and retrieval-only for test-time learning. Hybrid memory achieves the best test-time learning (28.9% with bounded accumulation at $C = 1024$, 27.0% with $C = 2048$ and $C = 8192$), substantially outperforming pure accumulation (20.7% with large context at $C = 8192$) and pure retrieval-only (11.4%). Pure retrieval cannot support learning because semantic search cannot capture temporal patterns or generalize across examples, while pure accumulation is weaker and much more expensive, requiring large context windows to approach hybrid performance. The hybrid advantage comes from retrieval providing historical context while bounded accumulation enables temporal pattern recognition and example-based generalization. Optimal accumulation size depends on model context window and should scale proportionally to model capabilities.

**M4. In-Session Learning With Hard Token Budget.** This justifies Design Law 6 in the main paper. When learning is required but token cost is constrained, accumulation (large context) is the only viable fallback, though it incurs substantial token cost that scales linearly with session length, making it unsuitable for long-running sessions.

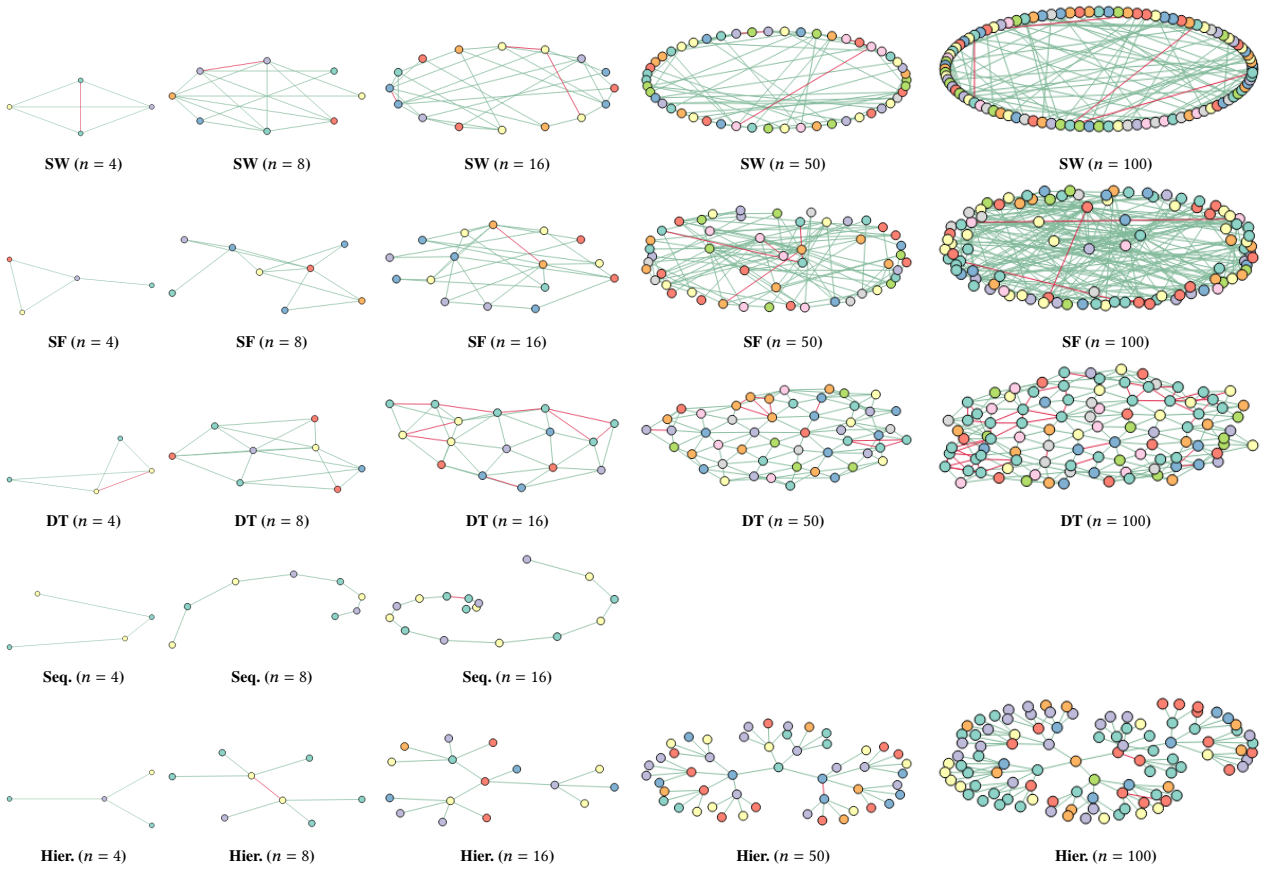


**Figure 5: Coloring experiment outcomes across different network sizes ($n = 4$ to 100). Each subfigure shows the final group assignments of agents. Valid assignments (neighbors in different groups) are shown in green, while conflicts (neighbors in the same group) are shown in red.**

Accumulation achieves 20.7% test-time learning at $C = 8192$, but incurs substantial token cost that scales linearly with session length. Hybrid memory incurs both retrieval and accumulation costs, making it unsuitable under strict budget constraints. This guideline explicitly encodes the accuracy–cost tradeoff: if cost is the primary constraint, accept lower accuracy (20.7% vs 28.9%) and use accumulation, scaling accumulation size with model context window capacity.

**M5. Long-Range Narrative Understanding.** This justifies Design Law 5 in the main paper. For systems requiring coherence across long documents or stories, retrieval-only achieves the best long-range understanding by preserving temporal ordering and narrative coherence. Pure retrieval-only achieves the best long-range understanding (30.4% with vector DB only), driven by strong performance on both summarization (20.0%) and narrative reasoning (40.8%). Hybrid designs actively degrade performance (17.6–25.4% across configurations) because mixing retrieved narrative fragments with accumulated context disrupts temporal structure, introducing competing story elements that interfere with long-range reasoning. Accumulation-based systems achieve moderate performance at large context sizes (27.5% with large context at $C = 8192$), but these gains come at substantially higher token cost and still fall short of retrieval-only accuracy.

**M6. Single-Hop Fact Revision (Limited Forgetting).** This justifies Design Law 7 in the main paper. For systems requiring updates to isolated facts, retrieval-only provides the best single-hop forgetting performance, substantially outperforming accumulation-based systems and hybrid designs. Retrieval-only achieves the best single-hop forgetting performance (36.0% FC-SH), substantially outperforming accumulation-based systems (0.0–0.4%) and hybrid designs (0.0–0.2%). However, even single-hop forgetting remains unreliable across all frameworks, indicating that explicit memory editing capabilities are necessary for reliable selective forgetting. This is a critical limitation of current architectures that developers should account for when designing systems requiring fact updates.

**M7. Multi-Hop Fact Revision.** This justifies Design Law 7 in the main paper. Multi-hop forgetting fails universally across all frameworks when revising dependent facts. Multi-hop forgetting (FC-MH) fails universally across all frameworks (<5% accuracy, with most achieving 0.0–0.5%). Hybrid memory exhibits the weakest behavior (0.0–0.2%) because it combines the failure modes of both paradigms—outdated facts persist in the retrieval store while obsolete information also remains in accumulated short-term context, making contradictions more frequent and harder to resolve. Since none of the evaluated frameworks supports explicit memory editing,

contradiction resolution, or targeted deletion, multi-hop selective forgetting is effectively unsupported. This is a critical negative result that highlights a fundamental limitation of current memory architectures.

**M8. Token Budget Must Remain Bounded.** This justifies Design Law 6 in the main paper. When cost must not grow with session length, retrieval-only maintains sublinear scaling and predictable latency, with runtime insensitive to session length. Retrieval maintains sublinear scaling and predictable latency, with runtime insensitive to session length. Accumulation scales linearly with session length and query frequency, with runtime dominated by prompt construction and relevance computation as context windows grow. High query workloads (100–200 questions per session) exhibit rapidly increasing runtime under accumulation, while retrieval maintains bounded cost. Hybrid designs incur both retrieval and accumulation costs, making them unsuitable when budget must remain strictly bounded.

**M9. Peak Accuracy Prioritized Over Cost.** This justifies Design Law 6 in the main paper. When accuracy matters more than tokens, hybrid memory (bounded, ∼512–1024 tokens; scale with model context window) provides the best balanced accuracy across competencies, jointly dominating accurate retrieval and test-time learning while maintaining competitive long-range understanding. Hybrid memory provides the best balanced accuracy across competencies, jointly dominating AR (44.9% with $C = 512$) and TTL (28.9% with $C = 1024$), while maintaining competitive long-range understanding (20.4–25.4%). Pure retrieval-only achieves a slightly higher overall score (23.8% vs 23.3%) by excelling in accurate retrieval (33.2%) and long-range understanding (30.4%), but fails on learning tasks (11.4%). The cost of hybrid memory is justified by superior performance across multiple competencies, making it the preferred choice when accuracy is prioritized. Scale context bounds proportionally to model capabilities.

**M10. Stable Runtime Across Long Sessions.** This justifies Design Law 6 in the main paper. When latency must not degrade over time, retrieval-only runtime is insensitive to session length, with minimal variation regardless of context size. Retrieval-only runtime is insensitive to session length, with minimal variation regardless of context size for tasks with few queries (1.6 average for long-range understanding). Accumulation degrades linearly with session length, as prompt construction and relevance computation costs grow with context windows. Hybrid designs inherit accumulation's linear scaling, making them unsuitable when stable runtime is required across long sessions.

**M11. Small Bounded Context Available (512–1024 Tokens).** This justifies Design Law 5 in the main paper. When only a small short-term context is available, hybrid memory (bounded) achieves optimal balance between accurate retrieval and test-time learning, with small bounded accumulation (512–1024 tokens) representing the sweet spot. This is the sweet spot for hybrid memory: $C = 512$ achieves the best accurate retrieval (44.9%), while $C = 1024$ achieves the best test-time learning (28.9%) and overall performance (23.3%). Performance degrades as accumulation grows beyond this range: accurate retrieval declines from 44.9% at $C = 512$ to 42.5% at $C = 2048$ and $C = 8192$, and long-range understanding degrades due to context mixing disrupting temporal structure. The relative pattern—small bounded accumulation outperforms large accumulation—generalizes across models, but absolute optimal sizes are model-dependent. Scale accumulation bounds proportionally to model context window capacity (e.g., for a 32K context model, use ∼128–256 tokens).

## E.2 Planning Architecture Guidelines

This section provides empirical justification for Design Laws 3–4 in the main paper. Planning experiments evaluate multiple models (7B–20B local, GPT-4.1/GPT-4o-Mini remote); optimal parameters and overhead multipliers vary with model capabilities.

**P1. Structured Task Decomposition.** This justifies Design Law 3 in the main paper. For systems requiring explicit task decomposition to improve reasoning, free-form planning preserves or improves accuracy relative to no planning, with benefits varying by task type and model size. For numerical reasoning (GSM8K), free-form planning helps small models substantially (Qwen-7B: 13.0% → 28.7%, Llama-3.1-8B: 65.6% → 71.9%) but may slightly degrade large models (GPT-OSS-20B: 94.6% → 87.6%). For commonsense reasoning (CSQA), free-form planning helps small models (Qwen-7B: 69.9% → 74.0%) while maintaining stability for large models. For symbolic reasoning (MATH-100), free-form planning improves across model sizes. Free-form planning introduces minimal overhead (1.2–2.5×) compared to schema-constrained planning (7–18×), which frequently degrades accuracy and introduces fragility. The relative pattern (free-form helps small models, schema-constrained is fragile) generalizes across models, but absolute improvements vary with model capabilities.

**P2. Model-Agnostic Robustness.** This justifies Design Law 3 in the main paper. For systems that must run reliably across different model families or sizes, free-form planning or no planning provides the most robust solution. Schema-constrained planning exhibits extremely high formatting failure rates for local models: Qwen-7B fails 84.7% on GSM8K, 41.0% on CSQA, and 70.0% on MATH-100; DeepSeek-7B fails 25.5% on GSM8K, 31.0% on CSQA, and 40.0% on MATH-100. These failures degrade accuracy by 10–40pp even when the model's reasoning is otherwise valid. Large remote models (GPT-4.1, GPT-4o-Mini) achieve 0.0% formatting failures with schema-constrained planning, but still incur 2–7× runtime overhead. Free-form planning maintains 0.0% failure rate across all models and datasets, ensuring stable performance regardless of model choice. If schema derivation is unavoidable (e.g., downstream systems require structured output), large models (>70B) can use schema-constrained planning with acceptable failure rates, but cost overhead remains substantial.

**P3. Small or On-Premise Models.** This justifies Design Law 3 in the main paper. For deployments with smaller models (<70B parameters), free-form planning or no planning is recommended. Schema-constrained planning incurs 7–18× runtime overhead (GPT-OSS-20B: 7.4× on GSM8K, 18.5× on CSQA, 11.4× on MATH-100) and systematic formatting failures for mid-sized models (Qwen-7B: 84.7% on GSM8K, 70.0% on MATH-100), making it unsuitable for resource-constrained deployments. Free-form planning enhances reasoning without fragility (Qwen-7B improves from 13.0% to 28.7% on GSM8K, 69.9% to 74.0% on CSQA) with modest overhead (1.2–2.5×), making it the preferred choice when planning is needed. No planning provides the lowest overhead (1×) and may be sufficient for simpler tasks or when accuracy gains from planning are marginal.

**P4. Latency-Critical Systems.** This justifies Design Law 4 in the main paper. When latency or throughput is prioritized over accuracy gains, no planning maintains the lowest latency with minimal token usage. Schema-constrained planning incurs 7–18× runtime overhead (GPT-OSS-20B: 7.4× on GSM8K, 18.5× on CSQA, 11.4× on MATH-100) and higher token usage, making it unsuitable for latency-critical applications. Free-form planning introduces modest overhead (1.2–2.5× runtime, moderate token increase), but no planning maintains the lowest latency with p50 <1s and minimal token usage. When latency is the primary constraint, disable planning entirely. If planning is required, free-form planning provides a cost-effective middle ground, trading 1.2–2.5× overhead for potential accuracy improvements.

**P5. Large Models with Schema Derivation Required.** This justifies Design Law 3 in the main paper. For systems using large models (>70B) where downstream systems require structured plan output, schema-constrained planning is viable but comes with substantial cost: 2–7× runtime overhead and higher token usage compared to free-form planning. Large remote models (GPT-4.1, GPT-4o-Mini) achieve 0.0% formatting failures with schema-constrained planning, making it viable when structured output is mandatory. Schema-constrained planning can even improve accuracy for some large models on specific tasks (GPT-4o-Mini: 86.5% → 90.8% on GSM8K). However, this comes at substantial cost: 2–7× runtime overhead (GPT-4.1: 2.4× on GSM8K, 2.9× on CSQA) and higher token usage compared to free-form planning (1.2–2.5×). Use schema-constrained planning only when structured output is unavoidable and cost overhead is acceptable. For all other cases, prefer free-form planning which provides similar benefits with lower cost.

## E.3 Topology and Coordination Guidelines

This section provides empirical justification for Design Laws 8–10 in the main paper. Topology experiments use GPT-OSS-20B; coordination behavior is topology-dependent across all evaluated models.

**T1. Global Consensus or Agreement Required.** This justifies Design Law 8 in the main paper. When all agents must converge to a single shared decision, centralized/broadcast (fully connected) topology is strictly dominant: it is the only topology class that succeeds across all network sizes, converging in exactly 3 rounds regardless of scale, with cost increasing predictably from 24k to 2,611k tokens and runtime from 26 to 922 seconds at $n = 100$. This behavior reflects one-hop broadcast where all agents observe identical global state at each round, eliminating fragmentation and delayed mixing. In contrast, graph-based topologies fail to achieve full consensus at scale despite substantially higher coordination cost: small-world graphs consume 10,664k tokens and require 15 rounds at $n = 100$ yet still fail; scale-free graphs reduce cost (5,734k tokens, 11 rounds) but also fail; and Delaunay graphs are both the most expensive (18,283k tokens, 19 rounds) and unsuccessful, as geometric locality prevents sufficient global information mixing.

**T2. Symmetry Breaking or Leader Election.** This justifies Design Law 10 in the main paper. For tasks requiring selection of a unique leader or breaking symmetry among equivalent agents, geometric/local (Delaunay) topology is the only graph-based structure that succeeds at leader election for $n \geq 8$, achieving convergence in 9–19 rounds but at substantially higher cost (up to 18,784k tokens and 1,463 seconds at $n = 100$), reflecting the expense of propagating leader confirmation through long communication paths. Small-world and scale-free graph-based topologies fail consistently across all network sizes despite increasing rounds and cost, indicating that diffusion-oriented connectivity does not provide a stable mechanism for global leader agreement. Role-based hierarchical orchestration succeeds only at small scale ($n = 8$) and degrades rapidly as depth increases, while fully connected topology succeeds only at $n = 4$ and fails thereafter. Effective leader election requires not just reachability, but a coordination structure that explicitly supports symmetry breaking rather than generic information diffusion.

**T3. Local Conflict Resolution (Neighborhood-Based).** This justifies Design Law 9 in the main paper. For systems where agents resolve conflicts using neighborhood information without requiring global coordination, diffusion-friendly (scale-free) topology achieves near-perfect success across all sizes (success = 1.0 at $n = 4$ and 0.97 at $n = 100$) while converging efficiently in 8–11 rounds, consuming 109k–5,699k tokens (runtime 66–652 sec). Small-world graphs also remain strong (success 0.83–0.98) but incur higher cost at large scale (125k–10,431k tokens, 72–929 sec runtime, 8–15 rounds). Geometric/Delaunay graphs show higher coordination burden and weaker stability at $n = 100$ (success 0.81, 19 rounds, 18,108k tokens, 1,209 sec). Role-based topologies (sequential/hierarchical) achieve perfect or near-perfect success at small $n$, but are not consistently available at larger $n$ under the same multi-round semantics. Scale-free topology leverages hub nodes to rapidly propagate conflict-resolution signals, making it optimal for local coordination tasks.

**T4. Bilateral Negotiation or Matching.** This justifies Design Law 9 in the main paper. For systems where agents engage in pairwise negotiation or matching without global constraints, diffusion-friendly (scale-free) topology provides the best cost-performance tradeoff. Scale-free and small-world topologies reach similar success at $n = 100$ (both 0.68), but scale-free is substantially cheaper: 5,274k tokens, 577 sec, 11 rounds versus small-world's 9,847k tokens, 910 sec, 15 rounds. Geometric/Delaunay graphs collapse at $n = 100$ (success 0.0) with very high cost (18,283k tokens, 1,400 sec, 19 rounds), indicating that geometric locality prevents reciprocal agreement from stabilizing globally. Centralized fully-connected topology maintains moderate success (0.42 at $n = 100$) with low rounds (3) but substantial cost due to dense messaging (3,016k tokens, 621 sec), reflecting that broadcast-like connectivity can accelerate agreement but does not guarantee correctness. Scale-free topology provides the best cost-performance tradeoff for bilateral negotiation tasks.

**T5. Global Constraint Satisfaction.** This justifies Design Law 9 in the main paper. When agents must satisfy global constraints requiring coordination across the entire network, diffusion-friendly (scale-free) topology achieves the highest success at large scale (0.85 at $n = 100$) with efficient coordination (11 rounds, 6,455k tokens, 670 sec). Small-world graphs reach moderate success (0.81 at $n = 100$) but require heavy coordination (15 rounds, 11,536k tokens, 1,057 sec), reflecting the iterative negotiation needed to satisfy global coverage constraints. Notably, scale-free performs worst at small network sizes (success 0.29 at $n = 8$ versus 0.79 for small-world), suggesting that hub dominance in small graphs can bias early coverage decisions and reduce solution quality under limited

rounds. Geometric/Delaunay graphs remain both expensive and less stable across scales (success 0.80 at $n = 100$, 18 rounds, 20,335k tokens, 1,516 sec), as geometric locality limits long-range coordination. Scale-free topology's hub-mediated diffusion accelerates global information flow, making it optimal for large-scale constraint satisfaction.

**T6. Large Agent Population ($n > 50$).** This justifies Design Law 9 in the main paper. For systems that must scale beyond 50 agents while maintaining performance, centralized/broadcast or diffusion-friendly (scale-free, small-world) topologies are recommended. Avoid geometric topologies due to diameter-induced degradation: Delaunay graphs show high coordination burden and degraded success rates (18–19 rounds, 18–20M tokens at $n = 100$). Centralized topologies remain viable for consensus tasks (3 rounds, 2,611k tokens at $n = 100$) but incur superlinear cost growth. Diffusion-friendly topologies (scale-free, small-world) sustain success rates >0.68 at large scale with moderate cost, but require dynamic reconfiguration to manage cost growth. Scale-free is most cost-effective (5,274–6,455k tokens at $n = 100$), while small-world incurs higher cost (9,847–11,536k tokens). All topologies show superlinear cost scaling, requiring careful cost management at scale.

## E.4 Framework Family Selection Guidelines

This section provides empirical justification for Design Law 1 in the main paper.

**F1. Deterministic Workflow with Low Overhead.** This justifies Design Law 1 in the main paper. For systems requiring deterministic workflow orchestration with minimal latency overhead, graph-based frameworks provide explicit execution graphs with fixed control flow, achieving near-native latency relative to direct LLM calls. Graph-based frameworks emphasize developer-driven orchestration through structured paths, with coordination fixed at design time.

**F2. Multi-Turn Agent Conversation Required.** This justifies Design Law 1 in the main paper. When systems require multi-turn conversation between agents, graph-based frameworks can support this through shared state along predefined edges, but require substantial developer code to implement conversation loops, state management, and message routing. Developers must explicitly define the graph structure, edge conditions, and state transitions to enable multi-agent dialogue.

**F3. Task-Driven Coordination with Role Specialization.** This justifies Design Law 1 in the main paper. For systems requiring task-driven coordination via role-specialized agents, role-based frameworks organize multi-agent systems around textual role specifications that condition agent goals, responsibilities, tool access, and interaction behavior. Coordination emerges through role-conditioned reasoning and task delegation rather than fixed workflows.

**F4. Delegation-Based Collaboration Needed.** This justifies Design Law 1 in the main paper. When systems require delegation-based collaboration where agents coordinate through task assignment and structured message exchange, role-based frameworks provide partial collaboration mechanisms through delegation-based coordination. Unlike graph-based frameworks which offer only procedural coordination, role-based frameworks enable agents to delegate tasks and coordinate dynamically.

**F5. Simulation of Emergent Behavior.** This justifies Design Law 1 in the main paper. For systems requiring simulation of emergent behavior through agent interaction, GABM (Generative Agent-Based Modeling) frameworks provide environment-mediated agent interaction where agents live in a shared world managed by a Game Master. This enables emergent, human-like group behavior but incurs extreme computational cost (tens of seconds latency for simple tasks).

**F6. Environment-Mediated Interaction.** This justifies Design Law 1 in the main paper. When systems require explicit world state with grounded variables that evolve over time, GABM frameworks provide environment-centric architectures where agents interact only via the Game Master, with no peer-to-peer communication. The environment maintains persistent state and mediates all agent interactions, enabling long-horizon emergent dynamics.

## E.5 Specialization and Environment Guidelines

This section provides empirical justification for Design Law 2 in the main paper.

**S1. Domain-Specific Reasoning.** This justifies Design Law 2 in the main paper. If task success depends on domain expertise, use procedural instructions (expert-guided conditioning) rather than role labels. Role-based prompting alone produces negligible performance differences relative to no-role baselines: on regression tasks, all roles achieve identical metrics (e.g., MAE 0.0669 across all roles on Utility), and on classification tasks, accuracy remains fixed (e.g., 45% across all roles on WiFi). Planning-based conditioning yields stable but tiny effects, with metrics differing by at most 0.1–0.2 points. In contrast, expert-guided conditioning produces large and consistent gains: accuracy increases from 45% to 60% on WiFi, from 38% to 44.83% on EU-IT, and reaches near-perfect levels (100%) on Yelp, with F1-scores increasing from 65% to over 95% on Volkert. These improvements are consistent across roles, indicating that methodological guidance, rather than identity framing, is the dominant factor driving expert-like behavior. Procedural instructions encode domain-specific workflows and constraints that activate latent domain knowledge in the model, whereas role labels provide only generic behavioral hints that do not reliably activate stronger domain-specific reasoning.

**E1. Grounded World State.** This justifies Design Law 2 in the main paper. If agents interact through shared state variables that evolve over time, deploy environment-centric architectures. Environment-mediated execution maintains persistent state across agent interactions, enabling stateful coordination and temporal consistency. However, environment-centric designs incur substantial overhead (10–50×) compared to stateless orchestration, as each agent action requires environment state updates and validation.

**E2. Emergent Behavior.** This justifies Design Law 2 in the main paper. If system behavior is expected to emerge over many rounds of interaction, environment-centric designs are necessary. Standard orchestration frameworks do not support long-horizon emergent dynamics because they lack persistent state and cannot model complex temporal dependencies. Environment-centric architectures enable agents to observe and react to evolving system state, facilitating emergent behaviors that arise from multi-round interactions.

### E.6 Model Dependency and Quantitative Results

All quantitative results reported in these guidelines are model-dependent and measured under specific experimental conditions. Memory experiments use GPT-OSS-20B (Groq-hosted) with ∼130K token context window; planning experiments evaluate multiple models (7B–20B local, GPT-4.1/GPT-4o-Mini remote); topology experiments use GPT-4o-mini. Optimal context sizes (e.g., 512–1024 tokens for hybrid memory), accuracy percentages, and cost multipliers will vary with model capabilities, context window limits, and token processing rates. The *relative patterns* and *architectural tradeoffs* (e.g., hybrid outperforms pure retrieval, scale-free excels for local coordination) are expected to generalize across models, but absolute numbers should be interpreted as model-specific measurements. When deploying with different models, adjust context budgets proportionally to model capabilities and validate performance empirically.

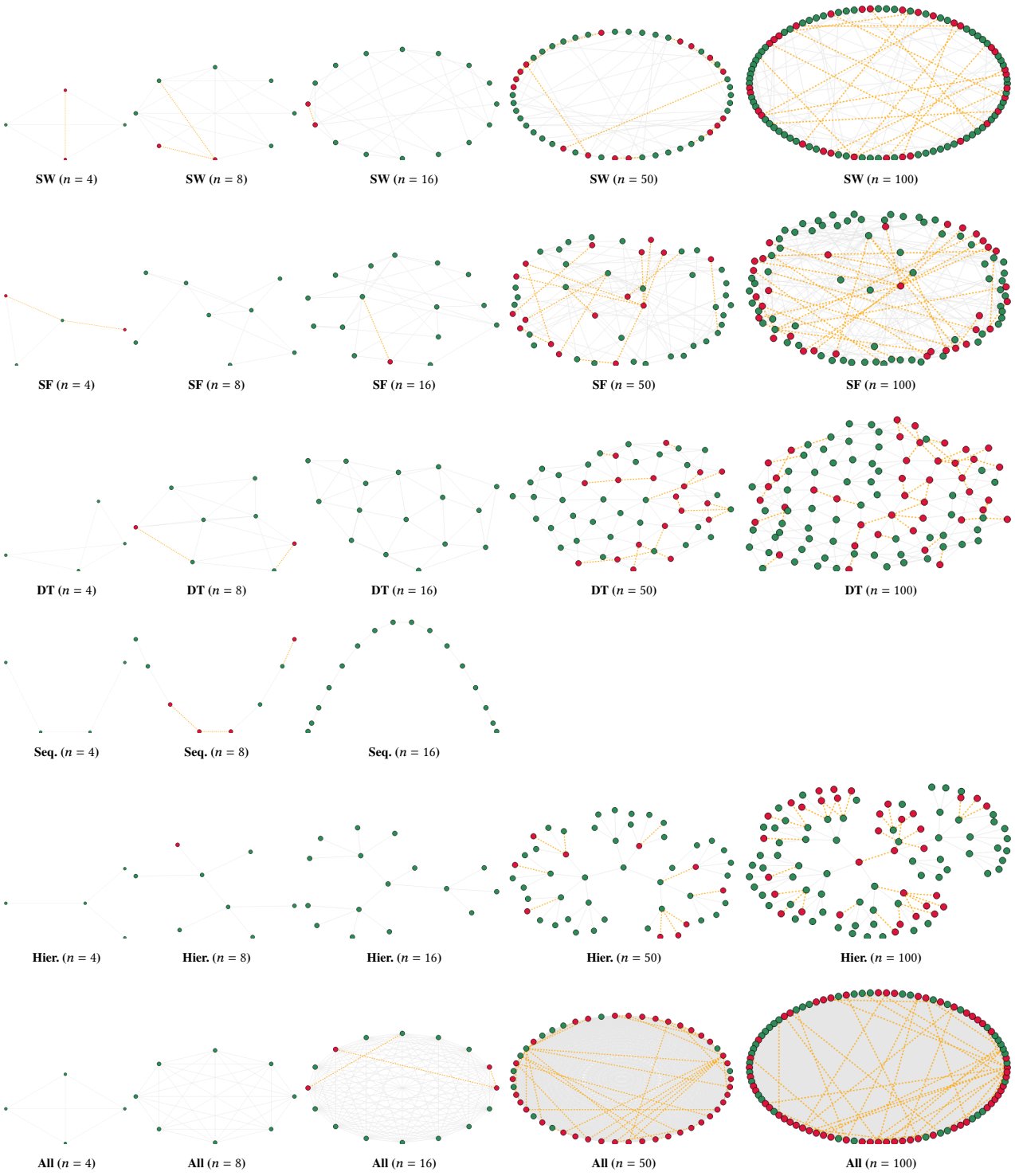


**Figure 6: Matching experiment outcomes using the *original scoring model*. Each agent selects one of its neighbors (or "None") to form a pair. Green nodes denote agents whose selections are locally consistent—that is, they selected a valid neighbor who reciprocated the choice. Red nodes represent agents involved in inconsistencies, such as choosing a non-neighbor, selecting a non-reciprocating partner, or forming idle pairs where both neighbors chose "None." The overall score corresponds to the fraction of locally consistent agents, following the computation described in the text.**

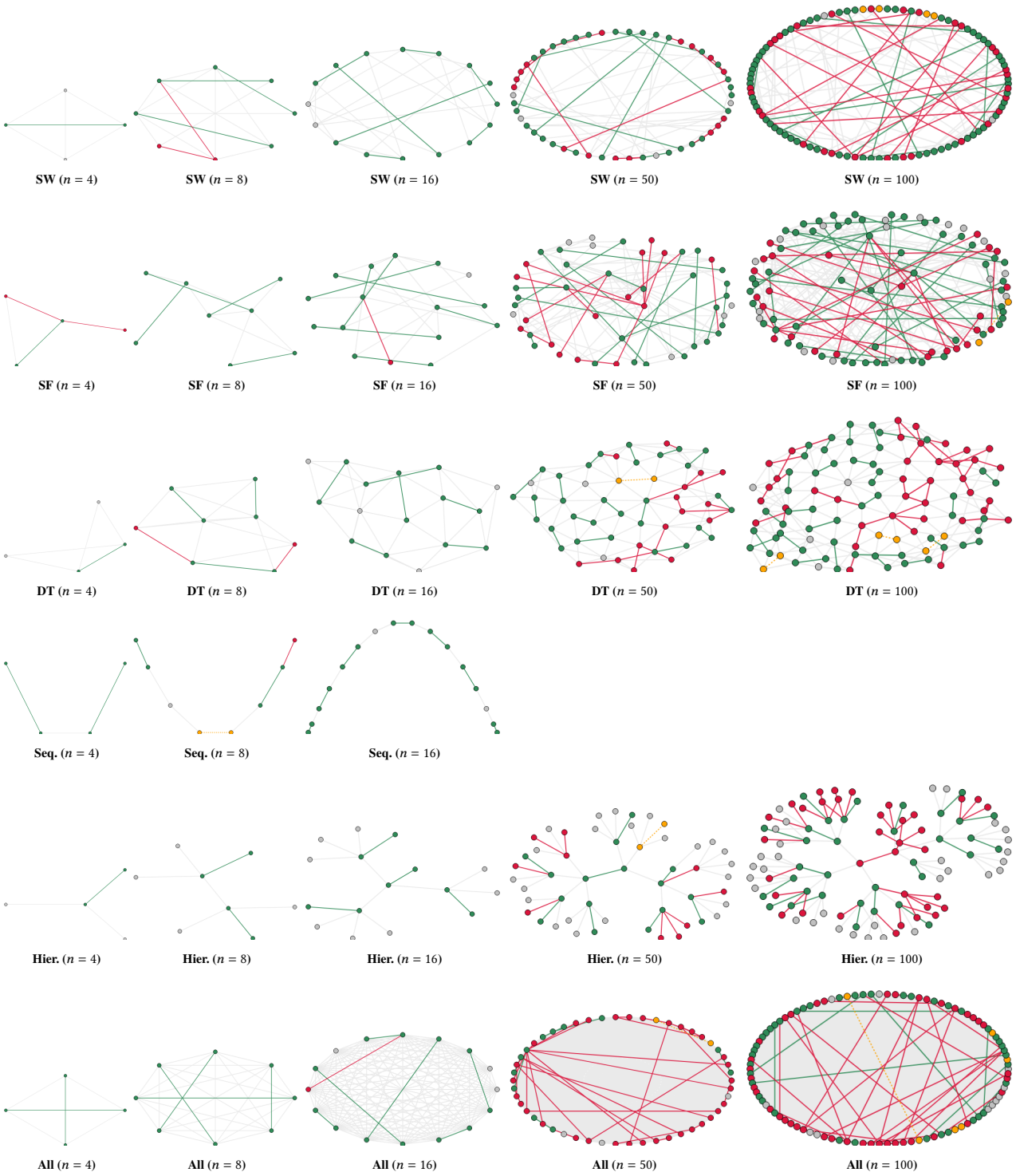


**Figure 7: Matching experiment outcomes across different network sizes ($n = 4$ to 100). Each subfigure shows the final pair assignments of agents based on the enhanced local-consistency model. Green edges connect mutually consistent agents whose choices satisfy the neighbor-reciprocation rule; orange dotted edges denote one-sided or invalid selections; and red nodes highlight agents involved in any local inconsistency. Gray edges represent structural links that did not participate in the matching process. This visualization extends the local scoring logic to explicitly reveal consistent and conflicting interactions.**

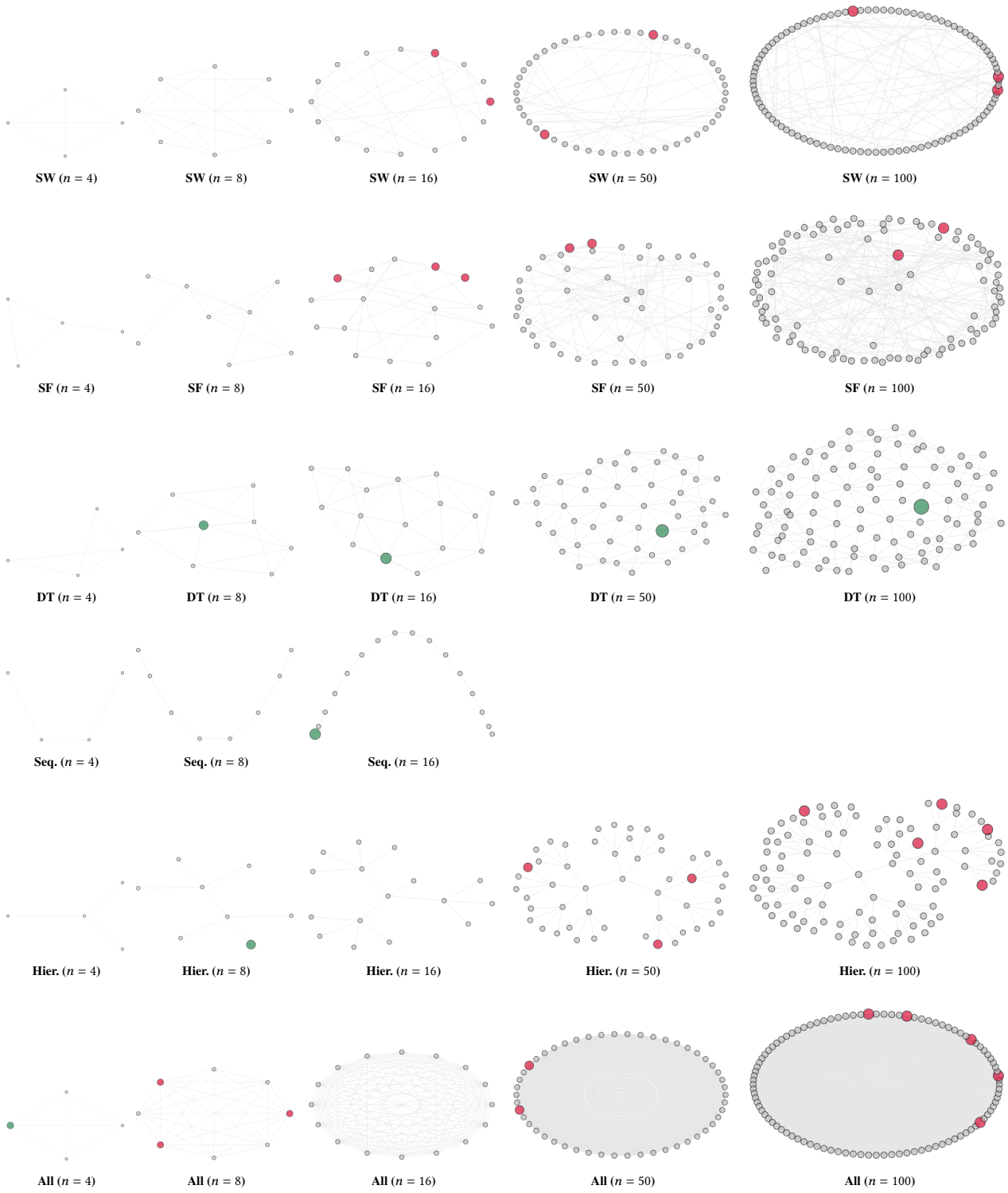


**Figure 8: Leader election results across different network sizes ($n = 4$ to 100). Each subfigure shows the final decisions of agents. Green nodes represent the correctly elected leader, red nodes indicate multiple leaders, and gray nodes represent followers.**

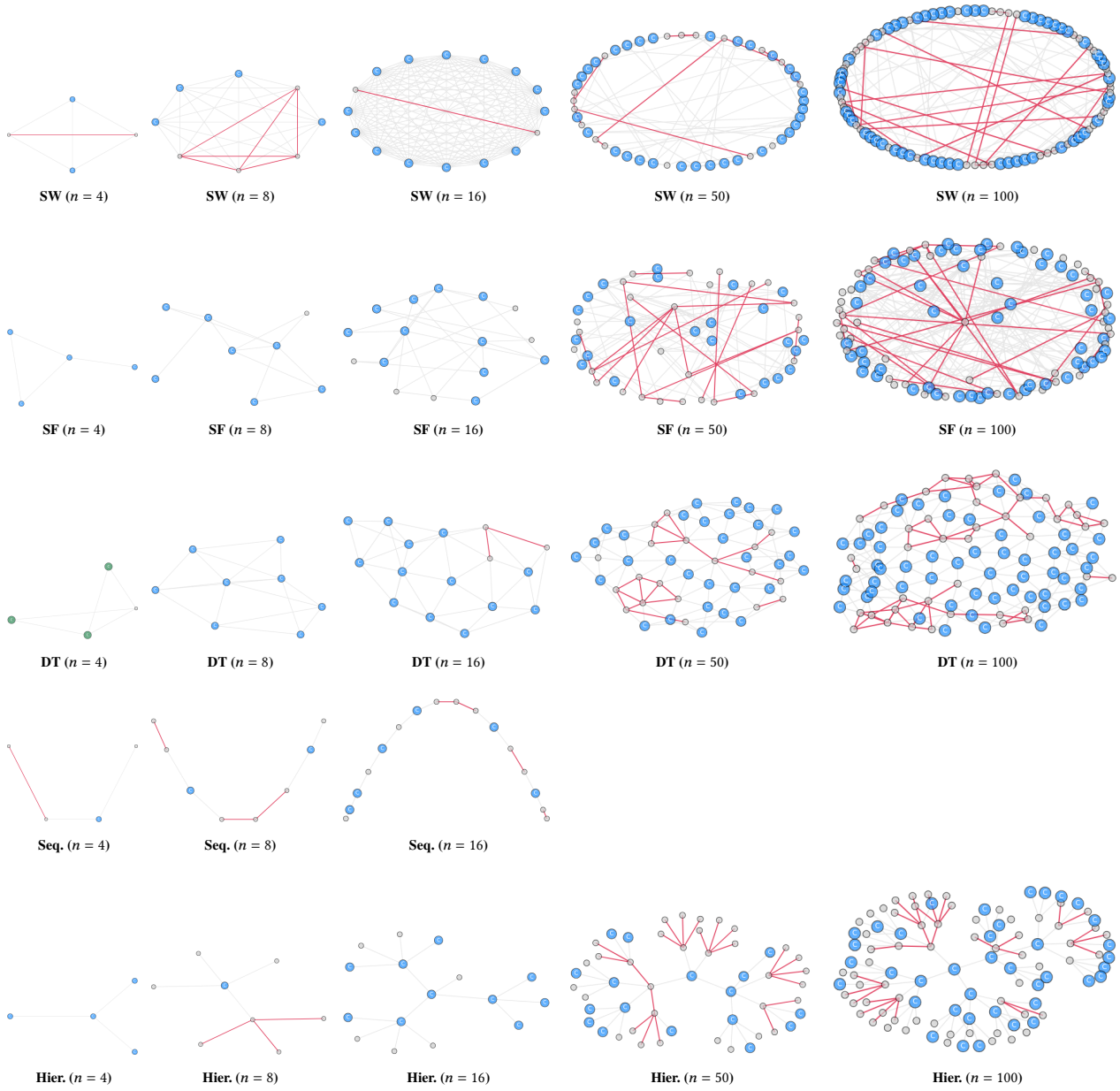


**Figure 9: Vertex cover results across different network sizes ($n = 4$ to 100). Each subfigure shows the agents' final decisions. Green nodes represent a minimal valid cover, blue nodes indicate members of a non-minimal cover, red edges mark uncovered pairs, orange nodes represent invalid responses, and gray nodes are non-cover agents.**